

UNIVERSIDAD NACIONAL DE AVELLANEDA



INGENIERÍA EN INFORMÁTICA

SISTEMAS DE CONTROL AUTOMÁTICO

AÑO 2022 – 2° Cuatrimestre

TRABAJO DE LABORATORIO: CONTROL PI DEL ELECTROVENTILADOR

Datos de los integrantes:

Apellido y Nombres

Micaela Gomez

Leandro Fenger

Lautaro Gaitan

Profesor/es: Guillermo Caporaletti - Juan Castellano

ÍNDICE

1. Objetivo	2
2. Introducción	2
2.1 Características del sistema:	2
2.2 Tipo de acción de control	3
2.3 Simulaciones	3
2.4 Mejora Software	11
3. Desarrollo	12
3.1 Mediciones	12
3.1.1 Medición con 500 RPM objetivo	12
3.1.2 Medición con 1500 RPM Objetivo	13
4. Conclusiones	16
5. Anexo I - Código Arduino	17
6. Bibliografía	27

1.Objetivo

El objetivo del presente trabajo es controlar el sistema físico (electroventilador) mediante una acción de control, empleando un controlador del tipo Proporcional Integrador para obtener la velocidad deseada, ver como él mismo se comporta frente a perturbaciones y si es capaz de estabilizarse coincidiendo con los valores que hemos obtenido en la teoría.

Hemos utilizado el mismo equipo que en la práctica del primer trabajo (“Control ON-OFF de un electroventilador”), en donde, el sistema es un electroventilador controlado por una placa Arduino, alimentado mediante un transistor Mosfet, con un optoacoplador que nos permite separar el circuito de control de la alimentación (12V 30A).

Se modificará el código ya existente, discretizando el valor de las RPM objetivo y modificando el T_i (Tiempo de integración) y K_P (Constante de proporcionalidad).

Esto nos permitirá concluir si lo que hemos calculado coincide con nuestro modelo físico y su comportamiento en la práctica.

2.Introducción

2.1 Características del sistema:

Realizamos el diagrama en bloques de nuestra planta que nos representará el sistema con el cual estamos trabajando y explicar cuál es nuestra variable manipulada y cual nuestra variable controlada.

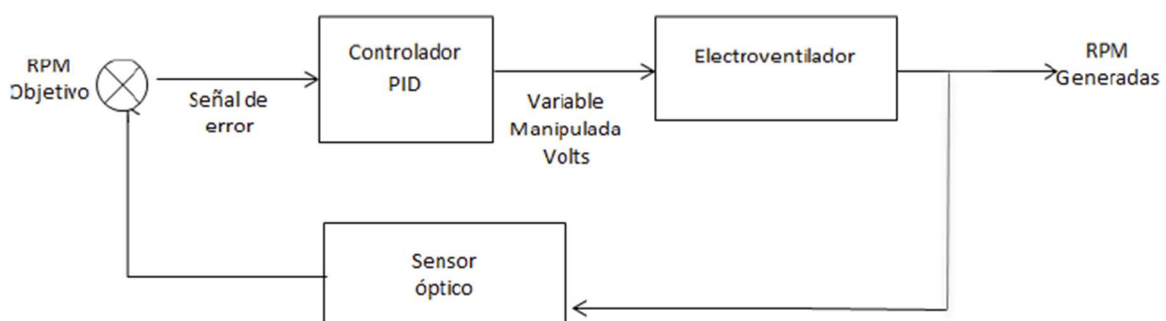


Fig. 1 - Diagrama de Bloques del Sistema Electroventilador.

Este es un sistema de control de lazo cerrado o realimentado, lo que nos significa que hay una variable controlada que es la cantidad o condición que se desea controlar, la cual se compara con el valor deseado y luego de esta se obtiene como resultado una señal de error. Esta señal de error es entregada al controlador que ejecutará una acción suficiente para modificar la variable controlada.

- **Planta:** es el electroventilador.
- **Variable de entrada:** RPM objetivo.
- **Variable controlada:** RPM generada por el electroventilador.
- **Variable manipulada:** La tensión ingresada al electroventilador.

Esta se manipula utilizando el ancho de pulso sobre el transistor que permite el paso de tensión hacia el electroventilador.

- **Controlador:** Hemos utilizado una placa Arduino programada en modo de acción de control proporcional integral con compensación.  sumado al controlador-actuador de tensión continua compuesto por un transistor mosfet y el optoacoplador.
- **Sensor Óptico:** Registra las mediciones de salida.

2.2 Tipo de acción de control

Para nuestro sistema, la acción de control es PI, en el cual se utiliza todo el rango de la señal manipulada, en nuestro caso de 0 a 12 V, para controlar la velocidad de la planta.

Esto se hace multiplicando la señal de error por un coeficiente de proporcionalidad (k_p) y combinando la acción del control integral que justamente integra esta señal de  control, la cual nos permite alcanzar los valores esperados con una mayor precisión.

2.3 Simulaciones

Para realizar las pruebas gráficas, utilizamos xcos.

Primero calculamos el Evans de la planta, el lugar geométrico de las raíces, donde se contempla el sistema a lazo abierto y evalúa cómo cambia el polo con una realimentación proporcional integral.

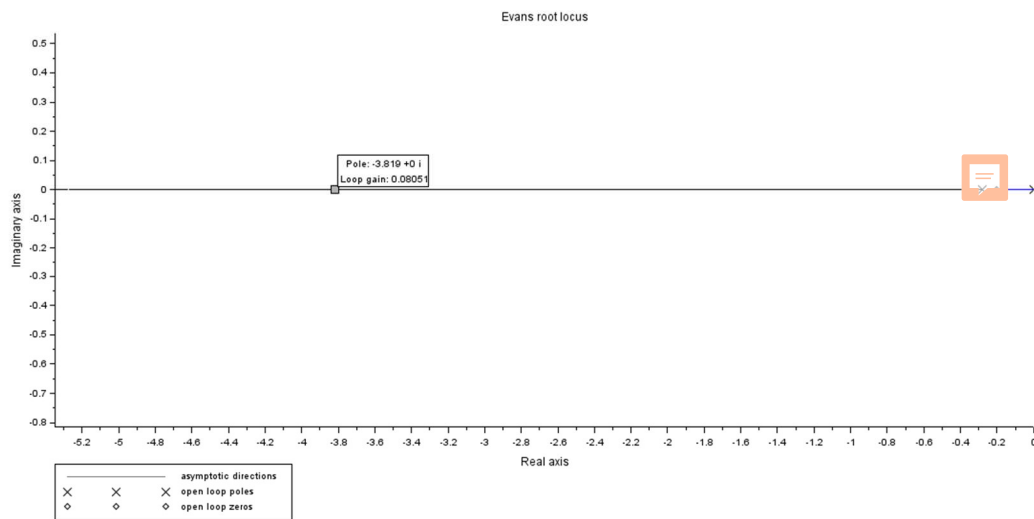


Fig 2. Gráfico y ubicación de polo, $K_p = 0,08$.

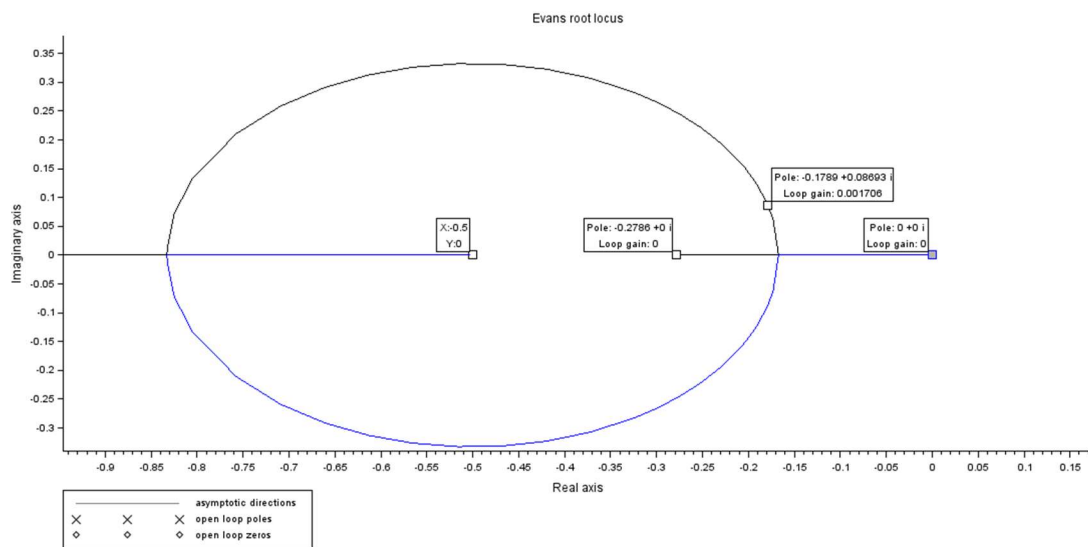


Fig 2.a Gráfico de Evans y ubicación de polos.

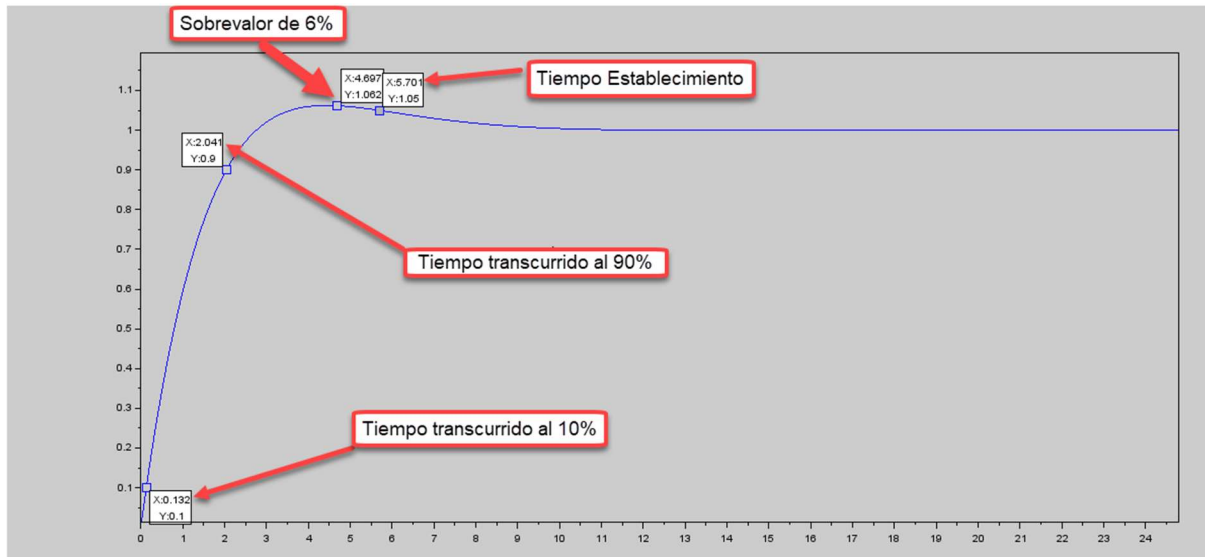


Fig 3. Gráfico de XCOS con $K_p = 0,017$.

Para el tiempo de crecimiento hemos establecido que tomaremos el tiempo transcurrido desde que se alcanza el 10% hasta el 90% del valor final.

Para el tiempo de establecimiento hemos seleccionado el tiempo transcurrido hasta que se alcanza un valor del $\pm 5\%$ del valor final.

Para el Tiempo de integración empezamos definiendo que, para analizar el comportamiento, este debería ser en primer lugar menor y luego mayor a nuestro tiempo natural del sistema ($\tau = 3,59$ s) para hacer las comparaciones y luego quedarnos con un valor.

La siguiente tabla muestra los resultados obtenidos en algunas pruebas que estuvimos realizando en las simulaciones, en donde en algunos casos hemos observado sobrevalores y que en base a los distintos valores obtenidos, nos permitirá definir el T_i y el K_p indicado que no genere sobrevalor y que se estabilice más rápido tras perturbaciones.

T_i	K_p	Sobrevalor	Tiempo Crecimiento(s)	Tiempo Establecimiento(s)
2	0,017	6%	1,90	5,70
2	0,005	4,40%	5,13	6,30
2	0,05	4,30%	0,77	0,97
5	0,017	-	3,76	6,35
5	0,08	-	0,62	0,92

Fig 4. Tabla de valores obtenidos tras simulaciones.

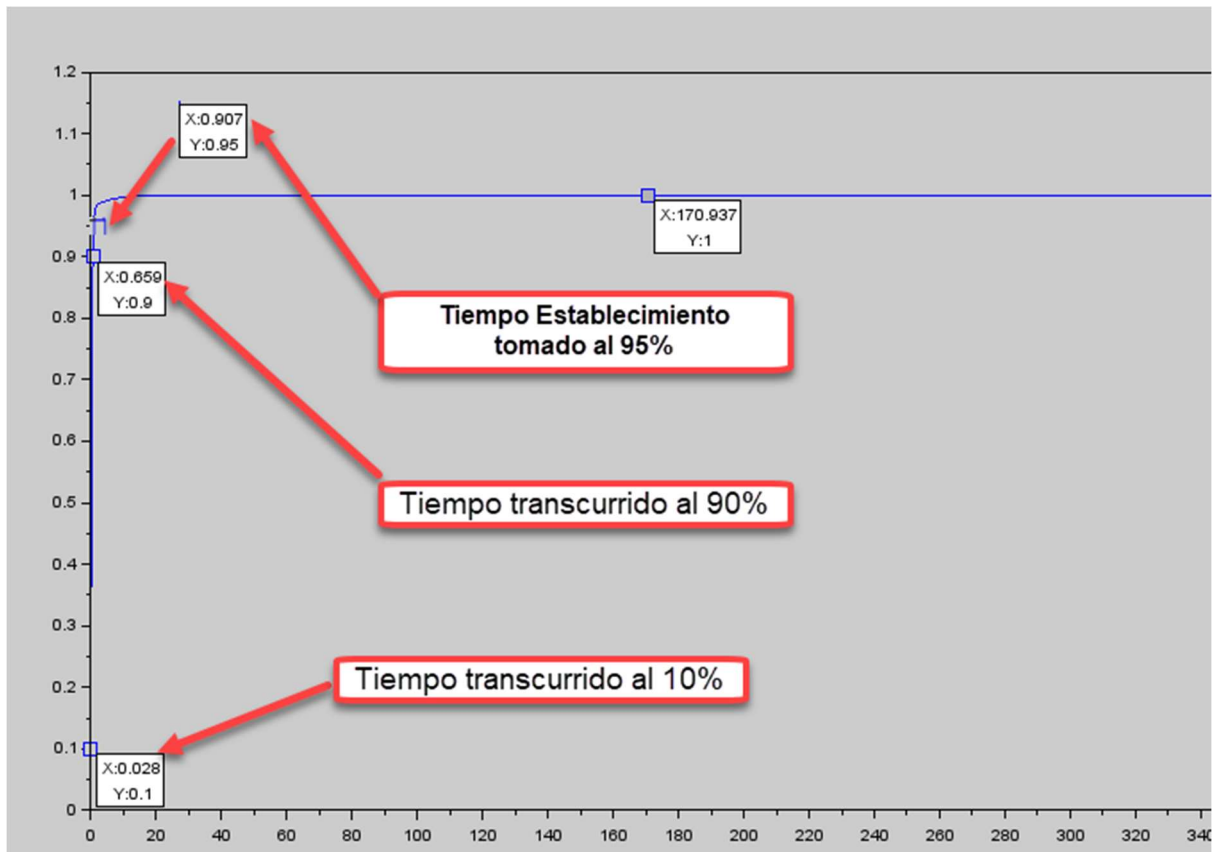


Fig 5. Gráfico en XCOS, $T_i = 5$ y $K_p = 0,08$.

Tras las simulaciones realizadas, determinamos que los valores que tomaremos para llevar a cabo en la práctica es un $T_i = 5$, un $K_p = 0,08$ en donde no tenemos sobrevalor a considerar y los tiempos de crecimiento y de establecimiento son aceptables.

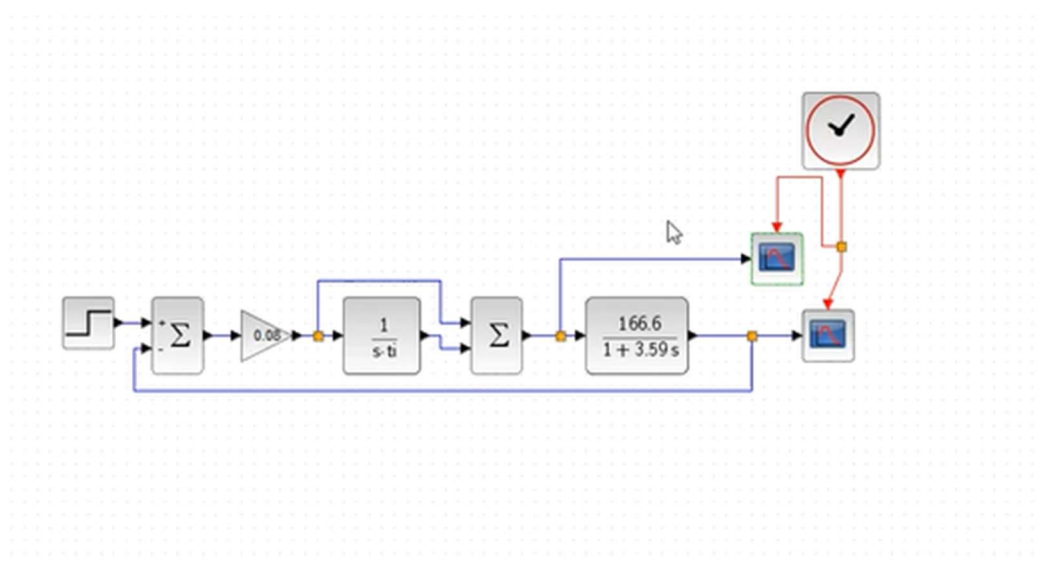


Fig 6. Modelo de Planta con Controlador Proporcional-Integral.

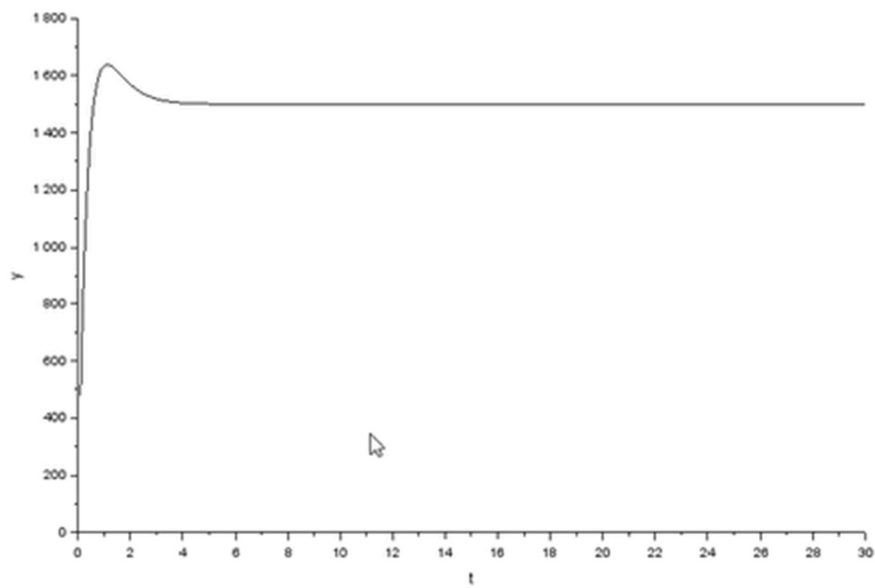


Fig 7. Respuesta del Sistema.

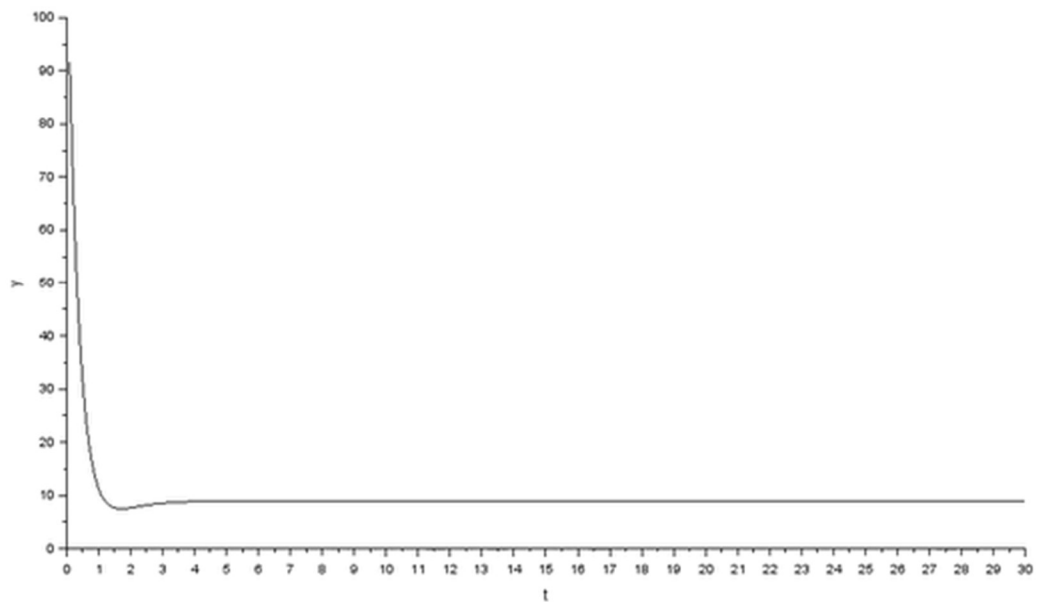


Fig 8 Respuesta del actuador.

En esta medición podemos observar que, en la respuesta del actuador, toma un valor de 90V cuando nuestro electroventilador funciona a 12V, lo cual nos llevó a considerar aplicar la saturación en la señal de entrada de la planta.

Simulación aplicando la saturación:

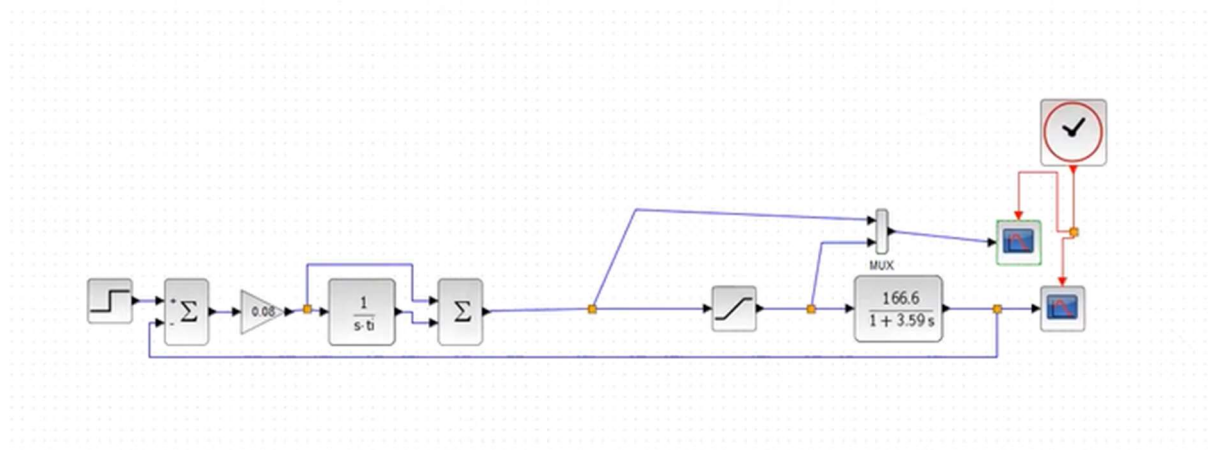


Fig 9. Modelo de Planta con Controlador PI y saturación del actuador.

Al simular la saturación del actuador podemos observar que a la salida del sistema se obtiene uno de los principales efectos indeseables de la saturación en un integrador que es un sobrevalor muy elevado debido a que el integrador continúa integrando mientras la entrada se encuentra saturada.

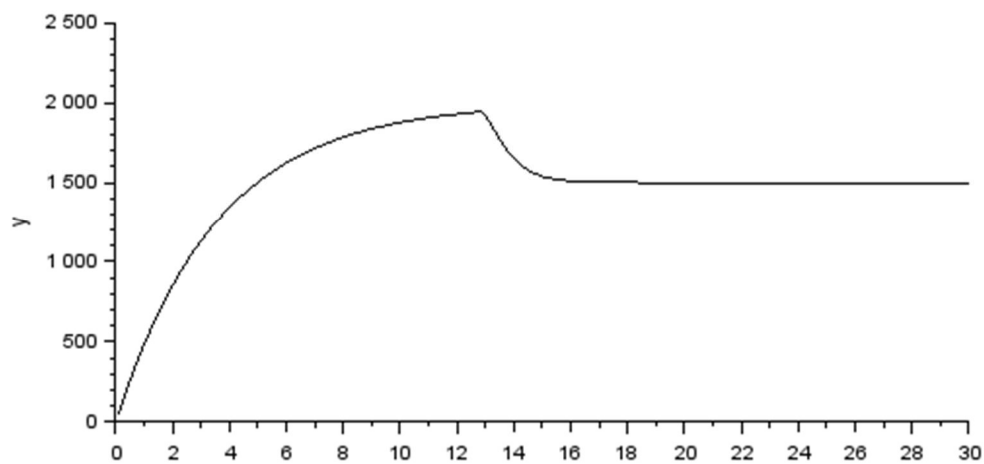


Fig 10. Salida del sistema con saturación y efecto enrole.

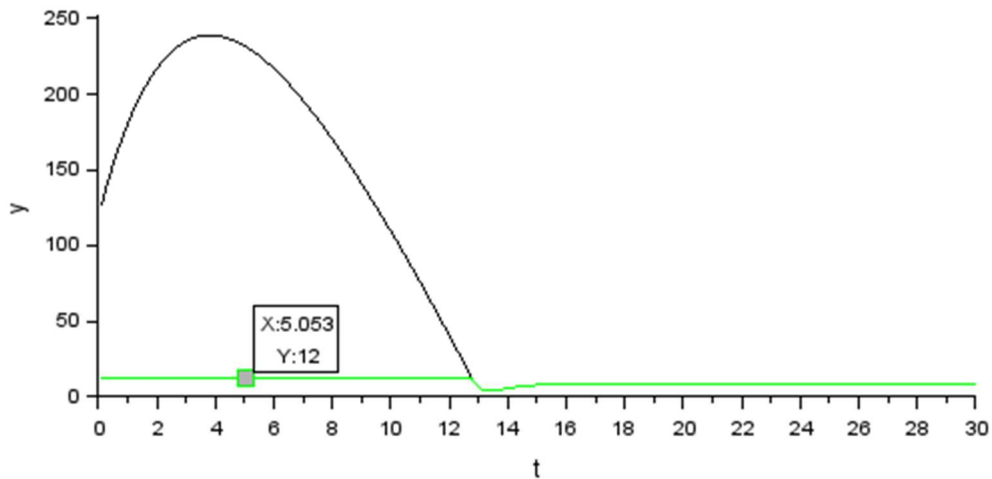


Fig 11. Salida del actuador con/ sin saturación y efecto enrole. 

La línea negra corresponde a la salida del actuador sin saturación y la línea verde hace referencia a la salida del actuador con saturación a 12V.

Podemos observar que aproximadamente a los 4 segundos comienza a bajar desde los 250V como consecuencia del integrador, luego de la saturación, aproximadamente a los 13 segundos baja la señal de control, lo que nos muestra que surge efecto la saturación del actuador, llevándolo a un valor estable de 12V.

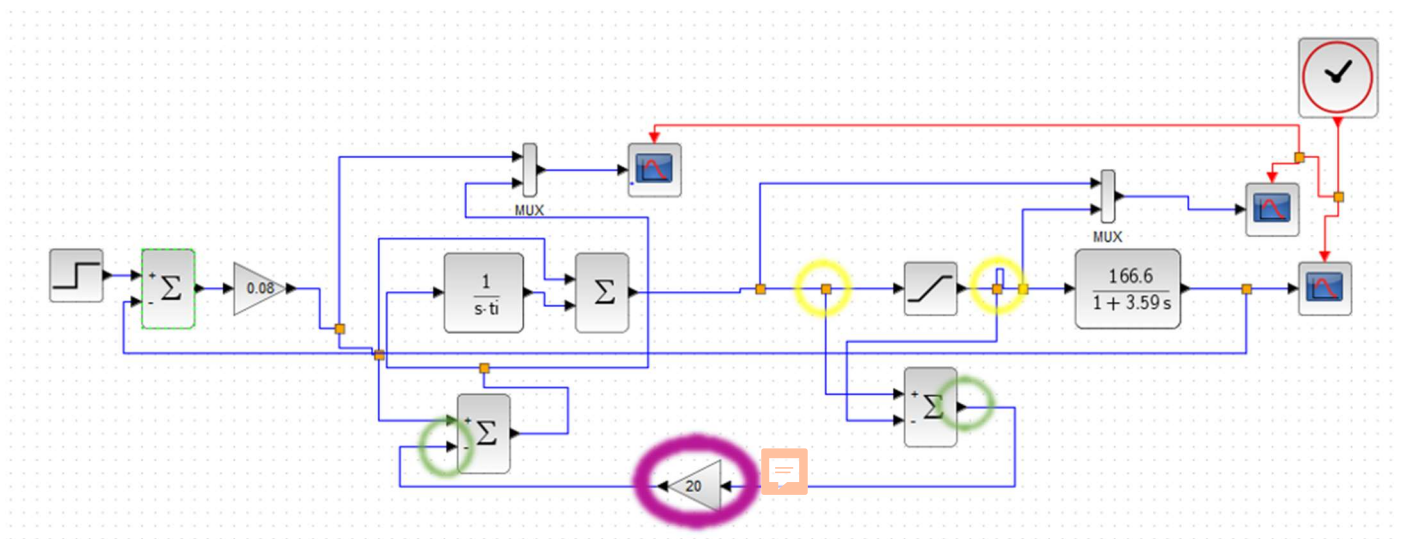


Fig 12. Modelo de planta con saturación y cor  i enrole.

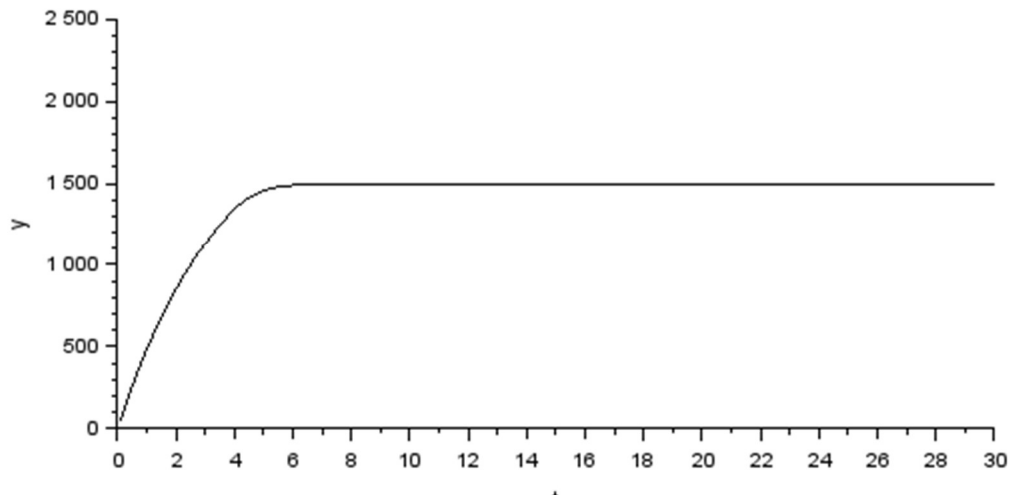


Fig 13. Salida del Sistema con anti enrole aplicado.

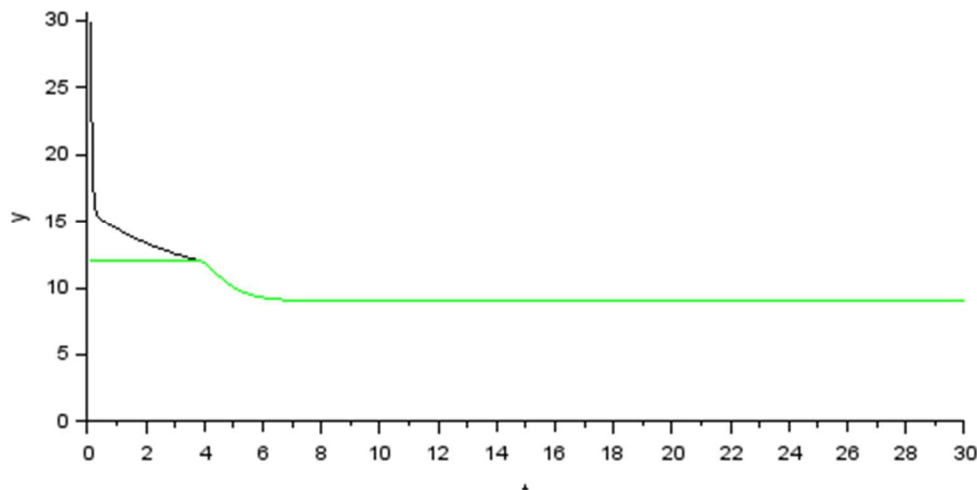


Fig. 14: Salida del actuado con anti enrole

Para compensar el efecto enrole debemos ponerle un freno al control integral cuando la diferencia entre la entrada y la salida del saturador (ver “Fig 12. Modelo de planta con saturación y con anti enrole”, referencias marcadas entre círculos de color amarillo) es distinta de cero, lo que significa que está actuando la saturación.

Si la diferencia entre la saturación es distinta de cero, se resta al error (ver “Fig 12. Modelo de planta con saturación y con anti enrole”, referencias marcadas entre círculos de color verde).

Al restar el error, la integral va a empezar a acumular negativamente y comenzará a restar área bajo la curva del error, entonces pasaremos a estar integrando el error hasta que haya una saturación.

Al momento que se detecta la saturación esa señal de error se resta para que no integre tanto.

Si le agrego una ganancia para que la saturación frente al error tome un valor significativo (ver “Fig 12. Modelo de planta con saturación y con anti enrole”, referencia marcada entre círculo de color violeta, por ejemplo, si tengo una diferencia de 3 entre las señales saturadas, lo multiplico por mi $k=20$, obtengo 60, luego a mi señal de error que por ejemplo es de 80 le resto esos 60, entonces solo voy a estar integrando 20).

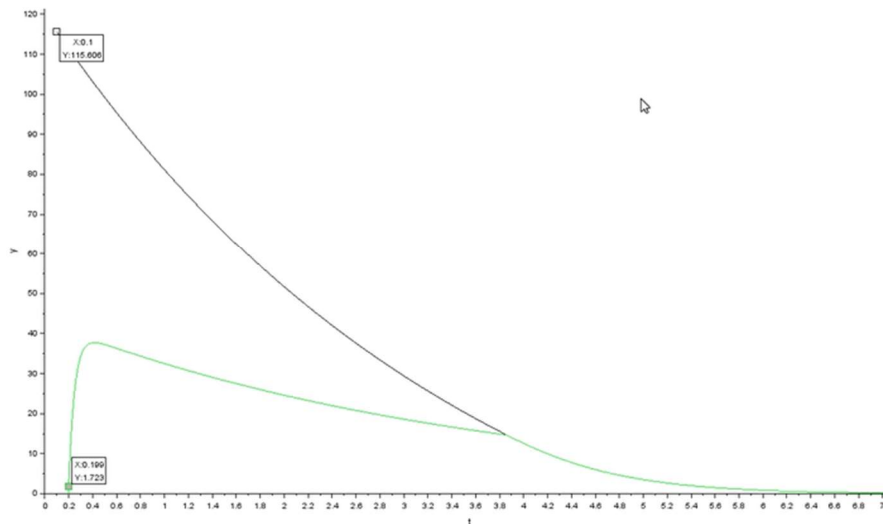


Fig. 15: Error con y sin arreglo del efecto enrollado.

En el gráfico vemos que la línea de color verde es el error modificado por el arreglo anti enrollado que es la entrada a nuestra parte integral y la línea negra es el error real, el que provoca el efecto enrollado, entonces logramos limitar el error real, evitando que crezca tanto la acción de control y con ello el efecto enrollado.

2.4 Mejora Software

Para llevar a cabo el desarrollo del código de programación se tuvieron en cuenta los códigos realizados por compañeros de cursadas anteriores agregando mejoras sobre los mismos. Además, se implementó la utilización de una librería elaborada en Sistemas de Control Automático denominada PID.h, allí se desarrolló la clase controlador que contaba con las propiedades y métodos más relevantes a utilizar con el objetivo de contrarrestar el efecto enrollado.

A su vez, se implementaron mejoras de software para evitar falsas lecturas frente a los cambios de estado ya que en el trabajo de laboratorio de control si-no, pudimos observar mediante los gráficos lecturas de picos muy elevados de RPM como consecuencia de falsas lecturas de cambio de estado, dichos cambios están determinados por las líneas blancas y negras del electroventilador. Realizamos una validación respecto al delta de tiempo que transcurre entre cada cambio de estado, de esta manera, si el tiempo detectado en un cambio de estado es menor al tiempo transcurrido previamente se descarta dicho cambio. Para ello, consideramos que el electroventilador tendrá 3000 RPM máximas, teniendo en cuenta que hay 4 cambios de estado por vuelta (dos líneas blancas y dos negras), 3000 RPM en 4 vueltas equivalen a 12000 cambios de estado por minuto. De esta manera, la frecuencia de cambios de estado será de 12000 por minuto, que evaluado en segundos corresponde a un valor de 200 por segundo, por lo cual, el tiempo mínimo entre cambios de estado que debemos considerar es de 1 entre 200, lo que nos da 0.005 segundos.



3.Desarrollo

3.1 Mediciones

3.1.1 Medición con 500 RPM objetivo

Objetivo: 500 RPM

Tiempo entre muestras: 50 ms

Nota: Se observa un sobrevalor de aproximadamente un 30%, lo cual no pudo percibirse en la teoría analizada previamente, en donde no obtuvimos sobrevalor.

También se ve como la compensación toma valores negativos siguiendo la curva de la acción proporcional.

Continuaremos analizando en otra medición y con otras RPM objetivo si el modelo de planta sugerido se ajusta a la realidad. A modo de hipótesis, entendemos que está relacionado con la alinealidad que presenta la saturación en el actuador, si bien intentamos contrarrestar el efecto enrollé, evidentemente en este caso no está sucediendo, teniendo en cuenta que cuando realizamos las simulaciones observamos un comportamiento similar en la señal de salida con un sobrevalor muy elevado debido a que el integrador del controlador continúa integrando mientras la entrada se encuentra saturada.

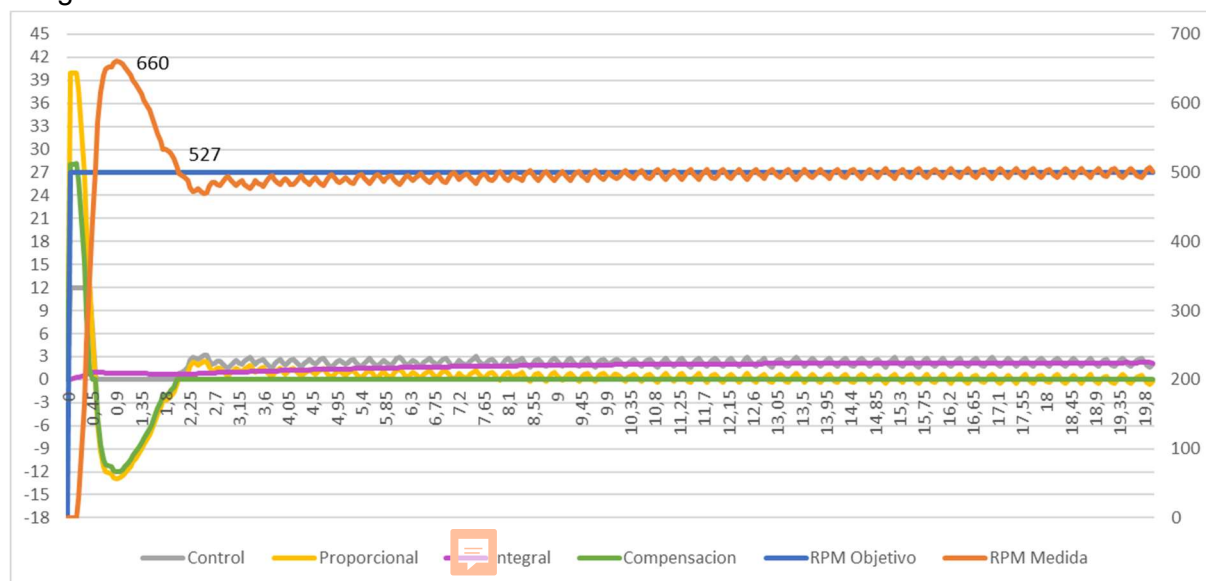


Fig. 16 Gráfico de Mediciones con 500 RPM.

En este gráfico podemos observar que las RPM Objetivo tienen un valor de 500, mientras que las RPM medidas tienden a ir acercándose a este valor. Ambos son referenciados al eje vertical derecho del gráfico expresado en RPM y al eje horizontal expresado en segundos.

Tiempo de Crecimiento:

Para el tiempo de crecimiento nosotros hemos elegido tomar el tiempo en estado transitorio que va desde que la señal alcanza un 10% hasta un 90% del valor final de nuestra salida.

Al estar al 10% en el crecimiento de la señal, las RPM se ubican en 50 y el tiempo en 230 ms, mientras que al alcanzar el 90%, las RPM son de 450 y el tiempo es de 450 ms.

Al realizar la diferencia, nos encontramos con un tiempo de crecimiento de 0,22 s 

Tiempo	Obj	Med
0	0	0
50	500	0
100	500	0
150	500	0
200	500	27
250	500	96
300	500	166
350	500	254
400	500	343
450	500	431
500	500	506
550	500	572
600	500	617
650	500	641

Fig. 17: Tabla de mediciones 500 RPM - Tiempo de crecimiento.

Para el tiempo de establecimiento nosotros hemos elegido un  valor de 5% del valor final.

Como en este caso hemos notado que tenemos un sobrevalor del 32%, consideramos tomar el tiempo para cuando la señal se ubica en un +5% de su valor objetivo (en nuestro caso 525 RPM aprox) obteniendo así un valor de 1.9 segundos.

Tiempo	Obj	Med
1600	500	568
1650	500	555
1700	500	545
1750	500	534
1800	500	534
1850	500	531
1900	500	527
1950	500	521
2000	500	508
2050	500	498
2100	500	496

Fig. 18: Tabla de mediciones 500 RPM - Tiempo de establecimiento.

3.1.2 Medición con 1500 RPM Objetivo

Objetivo: 1500 RPM

Tiempo entre muestras: 50 ms

Observaciones: Con Perturbaciones

Nota: En el siguiente gráfico no se observan sobrevalores respecto al valor de RPM objetivo lo cual se asemeja a la teoría y simulaciones analizadas previamente.

Tras haber transcurrido 12 segundos, introducimos una perturbación para ver la reacción de nuestro sistema frente a esta. En ese momento, el sistema intenta compensar dicha perturbación, aplicando la acción proporcional e integral. Una vez retirada la perturbación, se puede apreciar un muy pequeño sobrevalor que es rápidamente controlado debido a la acción proporcional.

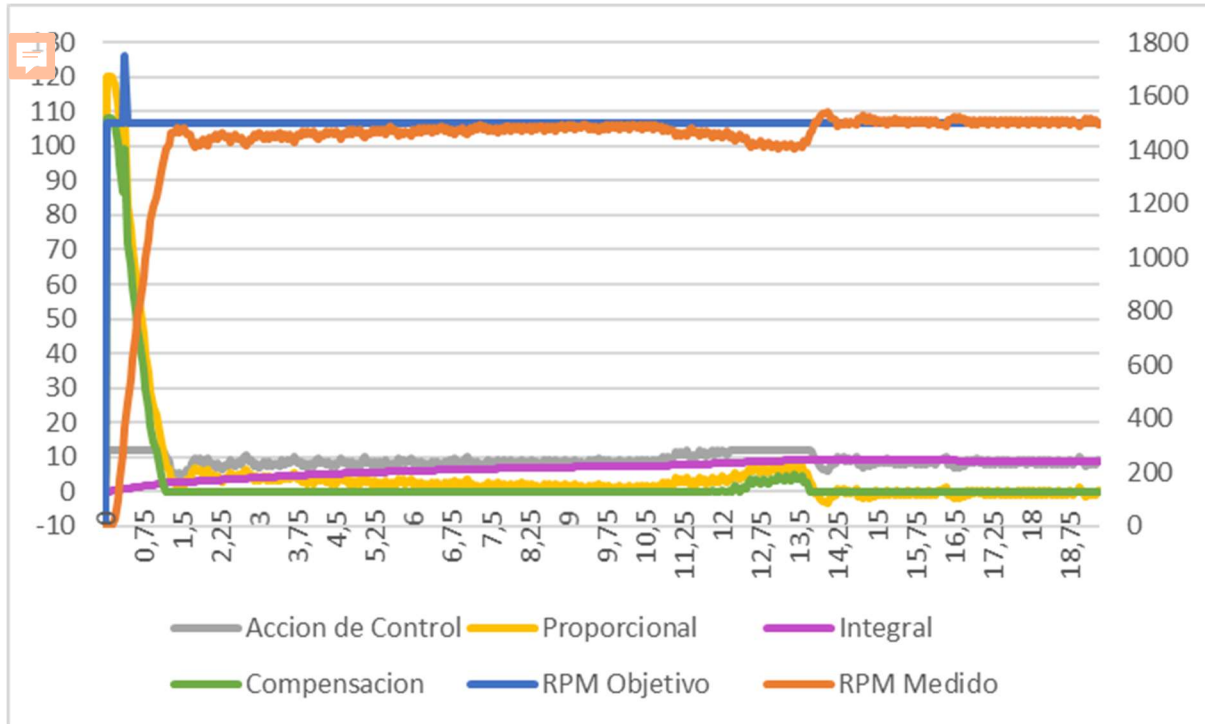


Fig. 19 Gráfico de Mediciones con 1500 RPM y perturbaciones.

Tiempo de Crecimiento:

Para el tiempo de crecimiento al igual que en la medición 1, hemos elegido tomar el tiempo en estado transitorio que va desde que la señal alcanza un 10% hasta un 90% del valor final de nuestra salida.

Al estar al 10% de la señal, medimos 150 RPM con un tiempo transcurrido de 290 ms.

Al alcanzar el 90%, medimos 1350 RPM, obteniendo un tiempo de 1150 ms.

Al realizar la diferencia, nos arroja el siguiente resultado.

$1150\text{ms} - 290\text{ms} = 860\text{ms} \rightarrow \mathbf{0.86s}$.

Tiempo	Obj	Med	Ctr	Tiempo	Obj	Med
0	0	0		900	1500	1134
50	1500	0		950	1500	1192
100	1500	0		1000	1500	1224
150	1500	0		1050	1500	1257
200	1500	25		1100	1500	1320
250	1500	98		1150	1500	1352
300	1500	178		1200	1500	1406
350	1500	273		1250	1500	1421
400	1750	369		1300	1500	1463
450	1500	461		1350	1500	1463
500	1500	547		1400	1500	1481
550	1500	622		1450	1500	1465
600	1500	712		1500	1500	1475
650	1500	777		1550	1500	1480
700	1500	845		1600	1500	1456
750	1500	926		1650	1500	1458
800	1500	1001		1700	1500	1432
850	1500	1070		1750	1500	1413

Fig. 20: Tabla de mediciones 1500 RPM - Tiempo de crecimiento.

Tiempo de establecimiento:

Para el tiempo de establecimiento nosotros hemos elegido un valor de 5% del valor final. Como en este caso hemos notado que no tenemos sobrevalor, consideramos tomar el tiempo para cuando la señal se ubica en un 95% de su valor objetivo (en nuestro caso 1425 RPM aprox) obteniendo así un valor de 1.25 segundos.

95% → 1425RPM → 1250ms → **1.25s**

	A	B	C
Tiempo	Obj	Med	
2	500	1500	547
3	550	1500	622
4	600	1500	712
5	650	1500	777
6	700	1500	845
7	750	1500	926
8	800	1500	1001
9	850	1500	1070
0	900	1500	1134
1	950	1500	1192
2	1000	1500	1224
3	1050	1500	1257
4	1100	1500	1320
5	1150	1500	1352
6	1200	1500	1406
7	1250	1500	1421
8	1300	1500	1463

Fig. 21. Tabla de mediciones 1500 RPM - Tiempo de establecimiento.

RPM objetivo	Sobrevalor	t crecimiento	t establecimiento 5%
500 RPM	32%	0.22s	1.9s
1500 RPM	0%	0.86s	1.25s

De esta manera, podemos observar que el sistema para 500 RPM si bien es más rápido debido a que posee un tiempo de crecimiento menos, posee un sobrevalor y como consecuencia, el tiempo de establecimiento es mayor. En cambio, con 1500 RPM se comporta como un sistema más lento debido a que su tiempo de crecimiento es mayor pero su tiempo de establecimiento es menor ya que no posee sobrevalor. 

4. Conclusiones

Tomando como punto a considerar que el objetivo de la práctica era controlar el electroventilador con una acción de control del tipo proporcional-integral, podríamos concluir que se ha logrado el mismo, aunque con algunas diferencias entre los resultados de las mediciones de la práctica y lo que habíamos calculado en la teoría. El control PI nos permitió manipular la constante de proporcionalidad y el tiempo de integración para modificar las magnitudes del control, en un primer instante, al no haber reunido las suficientes muestras de error para lograr un control integral se logra un control proporcional con las ventajas iniciales que éste ofrece (rápido crecimiento) y más avanzado en el tiempo, la acción de control se asemeja más a un control integral obteniendo un error de estado estacionario nulo. En conclusión, el controlador PI reúne lo mejor de las dos acciones de control, aunque en la práctica no obtuvimos resultados tan exitosos como esperábamos, en cuanto al tiempo de crecimiento observamos un notable cambio en ambas mediciones, por lo cual, podemos considerar posibles ventajas y desventajas de nuestro sistema planteado dependiendo el valor de RPM objetivo. En primer lugar, uno de nuestro objetivo fue contrarrestar la alinealidad del efecto enrolle mediante software, lo cual no se vió reflejado con un objetivo de 500 RPM, ya que se observó un sobrevalor muy elevado  en la medición se pudo observar todo lo contrario a lo esperado ya que a la salida del sistema se obtiene uno de los principales efectos indeseables de la saturación en un integrador que es un sobrevalor muy elevado debido a que el integrador continúa integrando mientras la entrada se encuentra saturada, en consecuencia el tiempo de establecimiento del sistema es mayor, por lo cual, la única ventaja que observamos para el objetivo de 500 RPM es un tiempo de crecimiento menor, lo que indica que el sistema será más rápido. Para un objetivo de 1500 RPM, logramos ver reflejadas en las simulaciones el correcto funcionamiento del efecto anti enrolle, debido a que no se observaron sobrevalores a la salida del sistema y en consecuencia el tiempo de establecimiento es menor, pero, el sistema es más lento respecto al objetivo de 500 RPM.. Uno de los puntos que logramos contrarrestar con éxito mediante software fueron los picos de RPM medidas producto de lecturas de cambios de estado erróneas, aunque el ruido aún continúa presente.

5. Anexo I - Código Arduino

Se realizaron mejoras sobre los códigos existentes en los trabajos de laboratorio de los cuatrimestres anteriores [2].

/******

Objeto para control PID

Sistemas de Control Automático (SCA)

Universidad Nacional de Avellaneda (UNDAV)

*****/

```
class controlPID          // Objeto para control Proporcional-Integral-Derivativo (PID)
{ private:
    float Salida;          // La señal de control que va al actuador
                          // o potencia de salida (sin asignar unidades)
    float Proporcional;    // Componente proporcional de la salida (sin asignar unidades)
    float Integral;        // Componente integral
    float Derivativo;      // Componente derivativa
    float Compensacion;    // Contiene la compensacion resultante ante la saturación
    float CompensacionAnterior; // Como usamos aproximación trapezoidal de la integral,
                          // necesitamos conservar el vaor anterior de compensación.
    float Kp;              // Constante proporcional (sin asignar unidades)
    float Ti;              // Tiempo de integración (en segundos)
    float Td;              // Tiempo para la componente derivativa (en segundos)
    unsigned long TiempoAnterior; // Tiempo de la medición anterior utilizando micros
    float ErrorAnterior;   // Señal de error anterior
    boolean LimitaSalida;  // Indica si establecimos límites superior e inferior a la
                          // salida
    boolean CompensaIntegral; // Indica si establecimos la compensacion de la integral
    float SalidaMax;       // Límite superior de la salida (y de la integral)
    float SalidaMin;       // Límite inferior de la salida
    const float MILLON=1e6; // Constante para convertir micros() a segundos.

public:
    controlPID(float KP, float TI, float TD); // Constructor con lo mínimo:
                          // KP: Constante de proporcionalidad (puede ser
                          // negativo)
                          // TI: Tiempo de integración (si es 0, no integra)
                          // TD: Tiempo de derivación (si es 0 no deriva)
    void ConfigurarPID(float KP, float TI, float TD); // Mismos parámetros que el
    constructor.
                          // Sirve para cambiar configuración inicial.
    boolean LimitarSalida(boolean RESPUESTA, float SMIN, float SMAX); // Configura los
    límites de salida.
                          // e indica si están activados.
    boolean LimitarSalida(boolean RESPUESTA); // Activa o desactiva los límites de
    salida
```

```

// e indica si el límite de salida está activado.
// No permite activar límites si antes no fueron
establecidos.
    boolean LimitarSalida();                // Indica si el límite de salida está activado.
    boolean CompensarIntegral(boolean RESPUESTA); // Activa o desactiva la
compensación de integración
// e indica si está activado.
    boolean CompensarIntegral();            // Me indica si la compensación está
activada.
    float Controlar(float ERROR);           // Calcula señal de control (salida) en
función del error.
    void Apagar();                          // Apaga el PID y resetea valores.
// No se modifican los valores de KP, TI y TD.
// Tampoco los límites pre establecidos.

    float ObtenerIntegral();
    float ObtenerProporcional();
    float ObtenerDerivativo();
    float ObtenerSalida();
    float ObtenerCompensacion();
};
/*****/

```

```

controlPID::controlPID(float KP, float TI, float TD)
// Constructor: incluye configuración inicial del PID y valores predeterminados.
{ // Kp puede ser negativo (esto último podría servir para controlar una planta cuya salida
// tienda a bajar cuando aumente la señal de control. Ej.: heladera.)
// Si Ti=0, el PID no lo tomará en cuenta
// Si Td=0, el PID no lo tomará en cuenta
// Inicializa integración e impone false en límites y compensación.
    ConfigurarPID(KP, TI, TD);
    LimitaSalida=false;
    CompensaIntegral=false;
}
//-----

```

```

void controlPID::ConfigurarPID(float KP, float TI, float TD)
// Configura las constantes básicas del control PID
// Sirve para cambiar configuración inicial, sin modificar límites y banderas.
{ // Kp puede ser negativo (esto último podría servir para controlar una planta cuya salida
// tienda a bajar cuando aumente la señal de control. Ej.: heladera.)
// Si Ti=0, el PID no lo tomará en cuenta
// Si Td=0, el PID no lo tomará en cuenta
    Kp=KP;
    Ti=TI;
    Td=TD;
// Resetea valores de integración.
    TiempoAnterior=0;
    ErrorAnterior=0;
}

```

```

    CompensacionAnterior=0;
    Integral=0;
}
//-----

boolean controlPID::LimitarSalida()
// Devuelve el valor de la variable privada LimitaSalida
// que indica si nuestro PID está configurado para limitar su salida.
{ return LimitaSalida;
}
//-----

boolean controlPID::LimitarSalida(boolean RESPUESTA)
// Configura si limitará la salida...
// No permite activar límites si antes no fueron establecidos.
{ LimitaSalida=RESPUESTA;
  if (SalidaMax==0 && SalidaMin==0) {
    // ...no voy a limitar porque no tengo límites definidos.
    LimitaSalida=false;
  }
  return LimitaSalida;
}
//-----

boolean controlPID::LimitarSalida(boolean RESPUESTA, float SMIN, float SMAX)
// Configura si limitará la salida entre SMAX y SMIN.
// Se puede establecer los límites pero no activarlos aún.
// No activa con SMIN=SMAX.
// No activa si SMIN>SMAX.
{ SalidaMax=SMAX;
  SalidaMin=SMIN;
  LimitaSalida=RESPUESTA;
  // La forma de desactivar este límite es:
  // 1) Volviendo a llamar esta función con RESPUESTA=false
  // 2) Llamando a LimitarSalida(false)
  // También se pueden poner límites muy grandes.
  if (SalidaMax==SalidaMin) {
    // ...no voy a limitar porque no tengo límites definidos.
    LimitaSalida=false;
  }
  if (SalidaMin>SalidaMax) {
    // ...no voy a limitar porque están mal configurados.
    LimitaSalida=false;
  }
  return LimitaSalida;
}
//-----

```

```

boolean controlPID::CompensarIntegral()
// Devuelve el valor de CompensaIntegral.
{ return CompensaIntegral;
}
//-----

boolean controlPID::CompensarIntegral(boolean RESPUESTA)
// Establece si debo compensar la integración cuando la salida está saturada.
// Deben haberse preestablecido los límites de salida.
{
    CompensaIntegral = RESPUESTA;
    if (!LimitaSalida) CompensaIntegral=false;
    return CompensaIntegral;
}
//-----

float controlPID::Controlar(float ERROR)
// Calcula Salida en función de la señal error y los parámetros del PID
{ unsigned long TiempoActual = micros(); // Tomo tiempo actual para comparar con la
medida anterior

    // PROPORCIONAL -----
    Proporcional = Kp*ERROR;

    // DERIVATIVO -----
    if (TiempoAnterior>0 && Td!=0) {
        // Dos condiciones para componente derivativa:
        // 1) Que no sea el primer cálculo y 2) Td seteado
        Derivativo = Kp*Td*(ERROR-ErrorAnterior)*MILLON / (TiempoActual-TiempoAnterior);
    } else {
        Derivativo = 0;
    }

    // ¿Debo compensar? -----
    Salida = Proporcional + Integral + Derivativo;
    Compensacion = 0;
    if (LimitaSalida && CompensaIntegral) {
        // Si no hay límite de saturación a la salida, no hay nada que compensar...
        if (Salida > SalidaMax) {
            // Debo saturar la salida porque supera el máximo...
            Compensacion = Salida - SalidaMax;
        }
        if (Salida < SalidaMin) {
            // Debo saturar la salida porque está por debajo del mínimo...
            Compensacion = Salida - SalidaMin;
        }
    }
}

```

```

// INTEGRAL -----
if (TiempoAnterior>0 && Ti!=0) {
    // Cumplidas las condiciones para integrar: (Si Compensacion==0, no va a compensar
nada...)
    Integral += (Kp * (ERROR+ErrorAnterior) - (Compensacion+CompensacionAnterior)) *
(TiempoActual-TiempoAnterior) / (2*Ti*MILLON);
    if (LimitaSalida) {
        // Debo saturar la integral (se supone que esto sólo podría pasar si cambio los
parámetros de integración...)
        Integral = min(Integral, SalidaMax);
        Integral = max(Integral, SalidaMin);
    }
}

// Termina componente integral -----

// Cálculo final completo:
Salida = Proporcional + Integral + Derivativo;

if (LimitaSalida) {
    // Debo saturar la salida:
    Salida = min(Salida, SalidaMax);
    Salida = max(Salida, SalidaMin);
}
TiempoAnterior = TiempoActual;
ErrorAnterior = ERROR;
CompensacionAnterior = Compensacion;
return Salida;
// Termina funcion PID -----
}
//-----

float controlPID::ObtenerIntegral()
{
    return Integral;
}
//-----

float controlPID::ObtenerProporcional()
{
    return Proporcional;
}
//-----

float controlPID::ObtenerDerivativo()
{
    return Derivativo;
}

```

```
//-----

float controlPID::ObtenerSalida()
{
    return Salida;
}
//-----

float controlPID::ObtenerCompensacion()
{
    return Compensacion;
}
//-----

void controlPID::Apagar()
{
    TiempoAnterior=0;
    ErrorAnterior=0;
    CompensacionAnterior=0;
    Integral=0;
    Proporcional=0;
    Derivativo=0;
}

/*****
 * CONTROL PI DE UN ELECTROVENTILADOR
 * =====
 *
 * Nuestro objetivo es controlar la velocidad de
 * un electroventilador de un auto.
 * El ventilador tiene dos barras blancas
 * y dos negras en toda su circunferencia.
 *
 *****/
#include "pid2.h"
const int pinActuador = 5; //pin hacia el transistor. Capacidad de PWM
const int pinSensor = 2; //pin del sensor
const int pinPotenciometro = A2; // pin del potenciometro
const int pinLedIntegrado = 13; // Pin del led que trae integrada la placa arduino
const int pinBotonParada = 7; // Pin del boton de parada de emergencia (chequear bien el
pin cuando se realice la conexion)
const int deltaT = 50; // Intervalo de tiempo entre muestras (en milisegundos)
const int maxRPM = 2000;

int objetivoRPM = 0;
int electroRPM = 0;
```

```

// Variables de manejo de la rutina de interrupción (ISR)
volatile unsigned long usAnterior=0; // Almacena el último tiempo de la interrupción en
microsegundos.
volatile unsigned long usDelta=0; // Almacena cuánto pasó entre cada interrupción en
microsegundos.
volatile unsigned long contadorISR=0; // Cuenta cambios de estado: otro modo de registro.
volatile bool estadoSensor; // Guarda estado último para evitar falsas detecciones de
cambios de estado.
long tiempoAnterior;
int velocidadAnterior = 0;
bool estadoBotonParada = false;

float Kp = 0.08; /*12; //kp se multiplica por 12 para cambiar la escala a Volt
float Ti = 5;
float Td = 0;
float SalidaMin= 0;
float SalidaMax= 12;
const bool limitarSalidaTrue = true; //
const bool CompensaIntegralTrue = true; //denota si se esta compensando la integral

controlPID controlador(Kp, Ti, Td);
float accionControl = 0;
float accionInt = 0;
float accionProp = 0;
float compensacion = 0;
int calculoActuador=0;

//*****
void setup() {
  pinMode(pinSensor, INPUT);
  pinMode(pinPotenciometro, INPUT);
  pinMode(pinActuador, OUTPUT);
  pinMode(pinBotonParada, INPUT);
  pinMode(pinLedIntegrado, OUTPUT);

  Serial.begin(57600);
  //attachInterrupt(digitalPinToInterrupt(pinSensor), interrupcion, CHANGE);
  attachInterrupt(digitalPinToInterrupt(pinSensor),ISRcalculaDelta,CHANGE);
  // Habilito las interrupciones el el pin 2.
  // Cuando detecte una interrupcion CHANGE el el pin 2,
  // se ejecuta la funcion interrupcion.
  tiempoAnterior = millis();

  //tiempo/velocidad objetivo/velocidad medida/acción de control (¿% ó V?) / parte
  proporcional /parte integral/compensación
  Serial.println( "Tiempo \tObj \tMed \tCtrl \tProp \tInt \tComp");
  mostrar(tiempoAnterior, objetivoRPM, electroRPM, accionControl, accionProp, accionInt,
  compensacion); //añadir variables

```



```

//Limito la salida del actuador y seteo sus valores min y maximos
controlador.LimitarSalida(limitarSalidaTrue, SalidaMin, SalidaMax);
controlador.CompensarIntegral(CompensarIntegralTrue);

// Muestro el estado inicial!!!
estadoSensor = digitalRead(pinSensor);
// Verifico inicialmente en qué estado está el sensor
// y lo guardo en la variable.
digitalWrite(pinLedIntegrado, LOW); // Apago el led.
}

//*****
void loop() {
  do {
    estadoBotonParada = digitalRead(pinBotonParada);

    // si se presionó el boton de emergencia --> paro el sistema
    if (estadoBotonParada)
    {
      Serial.print("Se ha accionado la parada de Emergencia");
      paramosSistema();
    }
  } while (millis() < (tiempoAnterior + deltaT));
  //leemos las RPM objetivos desde el potenciometro
  objetivoRPM = lecturaPotenciometro(pinPotenciometro);

  electroRPM = velocidadRPM(); // Leemos la velocidad del electroventilador

  //paso la diferencia entre las rpm leidas y las deseadas
  accionControl = controlador.Controlar(objetivoRPM - electroRPM); // Ajusta la tensión del
  ventilador en base a los RPM

  // accionControl = 9;

  //controlar si la accion de control da entre 0, 12
  //cambio la accion de control para que me de un ciclo de trabajo entre 0 y 255
  calculoActuador=accionControl*255/12;
  analogWrite(pinActuador, calculoActuador);

  tiempoAnterior = tiempoAnterior + deltaT; // Actualiza el tiempo del loop

  accionInt = controlador.ObtenerIntegral();
  accionProp = controlador.ObtenerProporcional();
  compensacion = controlador.ObtenerCompensacion();

  //actualizar mostrar.

```

```

    mostrar(tiempoAnterior, objetivoRPM, electroRPM, accionControl, accionProp, accionInt,
compensacion);
}
//*****
void paramosSistema()
{
    // 1) parar todo
    //controlador.Apagar();
    analogWrite(pinActuador,0);
    // 2) enviar info --> parpadea led
    // 3) loop infinito
    Serial.println("Boton de parada accionado.");
    do
    {

        digitalWrite(pinLedIntegrado, LOW);
        delay(1000);
        digitalWrite(pinLedIntegrado, HIGH);
        delay(500);
    } while (true);
}
//*****

void ISRcalculaDelta(){
// ISR que calcula cuánto pasó desde la anterior interrupción
//3000RPM * 4 VUELTAS = 12000
//12000/60= 200
//1/200 = 0.005SEG
if (digitalRead(pinSensor) != estadoSensor && micros() - usAnterior > 5000)
{
    estadoSensor=!estadoSensor; // Cambió el estado del sensor
    usDelta=(micros()-usAnterior); // Calcula delta (sin filtro promediador)
    contadorISR++; // Agrega cuenta
    // usDelta --> Almacena cuanto pasó entre cada interrupción en microsegundos.
    // usAnterior --> Almacena el último tiempo de la interrupción en microsegundos.
    usAnterior=micros(); // Actualiza
}
}

int velocidadRPM(){
    int med = 0;
// Devuelve la velocidad en RPM según el tiempo entre cambios de estado
if (usAnterior==0) {
    // Aún no hubo ninguna interrupción
    // Supongo una velocidad de 0 RPM
    med = 0;
}
//micros()--> es el tiempo que paso desde que se inicio el programa en microsegundos

```

```

//usAnterior --> el delta de la ultima interrupcion
//usDelta --> el tiempo q paso entre la interrupcion anterior y la actual
else if ((micros()-usAnterior)>usDelta) {
    // Quiere decir que la velocidad real es menor a la que indica msDelta
    // (Esto puede ocurrir porque msDelta sólo se actualiza cuando ocurre la interrupción)
    med = (15000000/(micros()-usAnterior));
} else
{
    // Calculo con msDelta :-)
    med = (15000000/usDelta);
}
med = min(med, 1.2*maxRPM);
med = 0.7*velocidadAnterior + 0.3*med;
velocidadAnterior = med;
return med;
}

```

```

//*****//
//tiempoAnterior, objetivoRPM, electroRPM, accionControl, accionProp, accionInt,
compensacion
void mostrar(unsigned long tiempo, unsigned int rpm, unsigned int measurerpm, float
accionControl, float accionProp, float accionInt, float compensacion) {
    Serial.print(tiempo);
    Serial.print("\t");
    Serial.print(rpm);
    Serial.print("\t");
    Serial.print(measurerpm);
    Serial.print("\t");
    Serial.print(accionControl);
    Serial.print("\t");
    Serial.print(accionProp);
    Serial.print("\t");
    Serial.print(accionInt);
    Serial.print("\t");
    Serial.println(compensacion);
}

```

```

int lecturaPotenciometro(int pinPotenciometro)
{
    // Lectura del potenciometro;
    int valorPotenciometro = analogRead(pinPotenciometro);
    // Mapeo de valor entre 0 y maxPRM
    int objetivoRPM = map(valorPotenciometro, 0, 1023, 0, maxRPM);

    //roundup to 50
    //objetivoRPM = round((float) objetivoRPM/50.0)*50;
    objetivoRPM = (( objetivoRPM+50) /250)*250;
}

```

```
// sacar promedio de potenciómetro roundup 50/100
if (objetivoRPM < 150) {
    objetivoRPM = 0;
}
return objetivoRPM;
}

// -----
```

6. Bibliografía

[1] Apunte “Margen de fase y margen de ganancia”, materia Sistemas de Control Automático, Undav.

[2] Novas Cristian, Onufryczuk Nicolas, Santari Alejandro (2021) Informe” Control PID de un electroventilador”, trabajo de laboratorio realizado en el curso de Sistemas de Control Automático, UNDAV.